

Guide de reprise par modules — MANRS West Africa Observatory

Ce document permet de reconstruire le projet de zéro, module par module, dans l'ordre. Chaque module est indépendant et livrable séparément à l'encadreur.

Vue d'ensemble

Module 1 – Base de données	← Fondation, à faire en premier
Module 2 – Collecteur de données	← Cœur du système, le plus complexe
Module 3 – API Backend	← Expose les données au frontend
Module 4 – Frontend React	← Interface utilisateur
Module 5 – Déploiement	← Docker + mise en ligne

Prérequis machine : Python 3.12+, Node.js 18+, Docker, Git

MODULE 1 — Base de données

Objectif

Créer les tables PostgreSQL qui stockent les ASN, les préfixes IP et les agrégats par pays.

Fichiers

Fichier	Rôle
backend/db/schema.sql	Script de création des 4 tables
docker-compose.yml	Conteneur PostgreSQL (section db : uniquement pour ce module)

Tables (4)

Table	Colonnes clés	Rôle
asn	asn_number, name, country_code, is_manrs_member, manrs_score, action_* (4 booleans)	Un rang par opérateur réseau
prefixes	asn_id (FK), prefix, roa_status	Préfixes IP annoncés par chaque ASN
countries	country_code, country_name, total_asn, roa_coverage_pct	Agrégats recalculés à chaque collecte
ai_recommendations	asn_id (FK), language, content	Cache IA (non utilisé dans la v1)

Fonctions à connaître

```
-- UPSERT : insérer ou mettre à jour sans doublon
INSERT INTO asn (...) VALUES (...)
ON CONFLICT (asn_number) DO UPDATE SET ...;

-- LATERAL JOIN : calculer des stats par ASN à la volée
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS total,
           COUNT(*) FILTER (WHERE roa_status = 'valid') AS valid
    FROM prefixes WHERE asn_id = a.id
) p ON TRUE
```

Points sensibles

- **CHECK constraint** sur manrs_score : doit être entre 0 et 4 sinon INSERT échoue
- **CHECK constraint** sur roa_status : doit être 'valid', 'invalid' ou 'not-found' exactement
- **ON DELETE CASCADE** sur prefixes.asn_id : si on supprime un ASN, ses préfixes sont supprimés automatiquement
- **INSERT des 16 pays** : le ON CONFLICT DO NOTHING évite l'erreur si on relance le script

Commandes pour ce module

```
# Lancer PostgreSQL en conteneur
docker compose up -d db

# Vérifier qu'il tourne
docker compose ps # doit afficher "healthy"

# Se connecter à la base
psql -U manrs -d manrs_observatory -h localhost

# Vérifier les tables
\dt
SELECT * FROM countries;
```

Critère de validation

Les 4 tables existent, les 16 pays sont insérés, les requêtes SQL de test passent.

MODULE 2 — Collecteur de données

Objectif

Un script Python qui interroge 5 sources de données et remplit la base toutes les 6h.

Fichiers (5)

Fichier	Rôle	Lignes
backend/collector/__init__.py	Package Python (vide)	0
backend/collector/config.py	Constantes : 16 pays, URLs des APIs, clé MANRS	34
backend/collector/apis.py	Fonctions d'appel aux 5 sources de données	245
backend/collector/db.py	Fonctions de persistance PostgreSQL	103
backend/collector/main.py	Orchestration : boucle pays → ASN → préfixes → score	144

Les 5 sources de données

#	Source	Fonction dans apis.py	Ce qu'elle retourne	Auth
1	MANRS v2 /ases	fetch_manrs_asns_for_country(country)	Liste des ASN d'un pays avec noms	Non
2	MANRS v2 /participants	fetch_manrs_participants()	Set des ASN membres MANRS	Clé API
3	RIPE Stat announced-prefixes	fetch_announced_prefixes(asn)	Liste des préfixes IP d'un ASN	Non
4	RIPE NCC RPKI Validator	validate_roa(asn, prefix)	Statut ROA d'un préfixe (valid/invalid/not-found)	Non
5	PeeringDB	fetch_peeringdb_info(asn)	Présence PeeringDB + IRR as-set	Non
6	CAIDA Spoofer	check_antispoofing(asn)	True/False/None (bloque le spoofing?)	Non (scraping)

Comment chaque action MANRS est évaluée

Action	Fonction	Logique	Résultat
Filtering	fetch_peeringdb_info()	has_irr = True si l'ASN a un irr_as_set renseigné	Bool
Anti-spoofing	check_antispoofing()	True si >50% des blocs IP testés bloquent le spoofing	Bool ou None

Action	Fonction	Logique	Résultat
Coordination	<code>fetch_peeringdb_info()</code>	<code>on_peeringdb = True</code> si l'ASN est présent sur PeeringDB	Bool
Validation	<code>validate_roa()</code>	True si >50% des préfixes ont un ROA valid	Bool
Score total	calculé dans <code>main.py</code>	Somme des 4 booleans (0 à 4)	Int

Fonctions dans `db.py`

Fonction	Rôle
<code>get_connection()</code>	Context manager : ouvre une connexion, commit à la fin, rollback si erreur
<code>upsert_asn(conn, data)</code>	INSERT ou UPDATE un ASN, retourne son id
<code>upsert_prefixes(conn, asn_id, prefixes)</code>	Supprime les anciens préfixes puis insère les nouveaux (batch)
<code>update_country_stats(conn, code)</code>	Recalcule <code>total_asn</code> , <code>manrs_members</code> , <code>avg_manrs_score</code> , <code>roa_coverage_pct</code> pour un pays

Flux d'exécution (`main.py`)

```

main()
  ↳ run_full_collection()
    ↳ fetch_manrs_participants() # charge les 1813 membres MANRS (une fois)
      ↳ pour chaque pays (16) :
        ↳ collect_asns_for_country(code, name, members)
          ↳ fetch_manrs_asns_for_country(code) # liste les ASN du pays
            ↳ pour chaque ASN :
              ↳ fetch_announced_prefixes(asn) # ses préfixes IP
              ↳ validate_roa(asn, prefix) # statut ROA de chaque préfixe
              ↳ fetch_peeringdb_info(asn) # coordination + filtering
              ↳ check_antispoofing(asn) # anti-spoofing
              ↳ upsert_asn(conn, data) # sauvegarde en base
                ↳ upsert_prefixes(conn, ...) # sauvegarde préfixes
            ↳ update_country_stats(conn, code) # recalcule les agrégats pays

```

Points sensibles

- **Rate limiting** : `time.sleep(0.5)` entre chaque appel API dans `_get()`. Sans ça, les APIs bloquent les requêtes.

- **try/except par ASN** : si un ASN échoue (API timeout, données corrompues), le collecteur continue avec le suivant. Ne JAMAIS faire planter le collecteur entier pour un seul ASN.
- **CAIDA Spoofer = scraping HTML** : la fonction `_load_spoof_data()` parse une page HTML avec des regex. Si CAIDA change le format de sa page, cette fonction casse. Le cache `_spoofer_cache` évite de recharger à chaque ASN.
- **Clé API MANRS** : expire après 3 jours. Variable d'environnement `MANRS_API_KEY`. Sans clé, le collecteur fonctionne mais `is_manrs_member` est toujours `False`.
- **Durée** : la collecte complète prend ~111 minutes pour 477 ASN. Le goulot d'étranglement est `validate_roa()` (1 appel par préfixe, 6810 préfixes au total).
- **fetch_abuse_contact()** : cette fonction est importée mais plus utilisée dans le flux actuel (la coordination se base uniquement sur PeeringDB). Elle reste disponible si besoin.

Commandes pour ce module

```
# Installer les dépendances
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Tester sur un seul pays (rapide)
DATABASE_URL="postgresql://manrs:changeme@localhost:5432/manrs_observatory" \
python3 -c "
from collector.apis import fetch_manrs_asns_for_country
asns = fetch_manrs_asns_for_country('BJ')
print(f'{len(asns)} ASN pour le Bénin')
for a in asns[:3]: print(f'  AS{a["id"]} - {a["name"]}')
"

# Lancer la collecte complète
MANRS_API_KEY="ta-clé-ici" \
DATABASE_URL="postgresql://manrs:changeme@localhost:5432/manrs_observatory" \
python3 -m collector.main
```

Critère de validation

Le collecteur tourne sans erreur pendant 24h. Les données des 16 pays sont en base. Les logs sont propres.

MODULE 3 — API Backend (FastAPI)

Objectif

Une API REST qui lit les données en base et les expose au frontend en JSON.

Fichiers (2)

Fichier	Rôle
backend/api/__init__.py	Package Python (vide)
backend/api/main.py	App FastAPI avec 5 endpoints

Endpoints (5)

Endpoint	Fonction	Ce qu'il retourne
GET /api/stats	get_stats()	{total_asn, manrs_members, avg_manrs_score, roa_coverage_pct}
GET /api/countries	list_countries()	Liste de 16 objets pays avec agrégats
GET /api/countries/{code}	get_country(code)	Détail pays + tableau de ses ASN
GET /api/asn/{number}	get_asn(number)	Fiche ASN complète : score, 4 actions, préfixes avec statut ROA
GET /api/search?q=	search_asn(q)	Recherche par numéro (exact) ou nom (LIKE)

Fonctions utilitaires

Fonction	Rôle
_query(sql, params)	Exécute une requête SQL, retourne une liste de dicts (via RealDictCursor)
_query_one(sql, params)	Comme _query mais retourne un seul dict ou None

Format de réponse — /api/asn/{number}

```
{
  "asn_number": 28683,
  "name": "SOCIETE BENINOISE D'INFRASTRUCTURES NUMERIQUES",
  "country_code": "BJ",
  "is_manrs_member": false,
  "manrs_score": 2,
  "actions": {
    "filtering": false,
    "anti_spoofing": false,
    "coordination": true,
    "validation": true
  },
  "prefixes": [
    { "prefix": "137.255.1.0/24", "roa_status": "valid" },
    { "prefix": "196.46.153.0/24", "roa_status": "not-found" }
  ]
}
```

```
],  
  "roa_coverage_pct": 92.59,  
  "last_updated": "2026-06-24T04:41:16"  
}
```

Points sensibles

- **CORS** : `allow_origins=["*"]` obligatoire car le frontend tourne sur un port différent (3000 vs 8000). En production, restreindre à l'URL du frontend.
- **RealDictCursor** : retourne les lignes SQL comme des dicts Python (`{"column": value}`), pas des tuples. Facilite la sérialisation JSON.
- **LATERAL JOIN** : utilisé dans `get_stats()` et `get_country()` pour calculer la couverture ROA à la volée au lieu de la stocker. Avantage : toujours à jour. Inconvénient : plus lent sur de gros volumes.
- **HTTPException(404)** : retourné quand un pays ou ASN n'existe pas en base. Le frontend doit gérer ce cas.
- **code.upper()** : le code pays est normalisé en majuscule côté API (bj → BJ).

Commandes pour ce module

```
# Lancer l'API  
source venv/bin/activate  
DATABASE_URL="postgresql://manrs:changeme@localhost:5432/manrs_observatory" \  
uvicorn api.main:app --reload --port 8000  
  
# Tester les endpoints  
curl http://localhost:8000/api/stats | python3 -m json.tool  
curl http://localhost:8000/api/countries | python3 -m json.tool  
curl http://localhost:8000/api/asn/28683 | python3 -m json.tool  
curl "http://localhost:8000/api/search?q=MTN" | python3 -m json.tool  
  
# Doc interactive auto-générée  
# Ouvrir http://localhost:8000/docs dans le navigateur
```

Critère de validation

Les 5 endpoints répondent correctement. Le format JSON est conforme. La doc Swagger (/docs) fonctionne.

MODULE 4 — Frontend React

Objectif

Un tableau de bord interactif avec carte, graphiques et fiches détaillées.

Stack

Technologie	Rôle
React 19 + TypeScript	Framework UI
Vite 8	Build tool (remplace CRA qui est deprecated)
shadcn/ui	Composants UI (Card, Table, Badge, etc.)
Phosphor Icons	Icônes (remplace les emojis)
React-Leaflet	Carte interactive Afrique de l'Ouest
Recharts	Graphiques (barres, camemberts)
React Router DOM	Navigation entre pages

Architecture Atomic Design

frontend/src/	
├── types/api.ts	← Interfaces TypeScript (contrats API)
├── lib/	
│ ├── api.ts	← Client HTTP (fetch vers FastAPI)
│ └── countries.ts	← Noms des 16 pays + helpers couleurs ROA
├── hooks/	
│ └── use-fetch.ts	← Hook générique fetch avec loading/error
├── components/	
│ ├── ui/	← Composants shadcn (NE PAS MODIFIER)
│ │ ├── badge.tsx	
│ │ ├── button.tsx	
│ │ ├── card.tsx	
│ │ ├── input.tsx	
│ │ ├── progress.tsx	
│ │ ├── separator.tsx	
│ │ └── table.tsx	
│ ├── atoms/	← Composants unitaires
│ │ ├── stat-value.tsx	← Carte stat avec icône et valeur
│ │ ├── roa-badge.tsx	← Badge coloré valid/invalid/not-found
│ │ ├── action-icon.tsx	← Case ✓/x pour une action MANRS
│ │ └── loading.tsx	← Spinner + message d'erreur
│ ├── molecules/	← Combinaisons d'atomes
│ │ ├── stats-banner.tsx	← 4 stat cards (ASN, MANRS, Score, ROA)
│ │ ├── search-bar.tsx	← Input de recherche dans le header
│ │ ├── manrs-scorecard.tsx	← Barre de score 0-4 + 4 action icons
│ │ └── roa-gauge.tsx	← Camembert ROA avec légende
│ ├── organisms/	← Composants complexes
│ │ ├── west-africa-map.tsx	← Carte Leaflet avec cercles cliquables
│ │ ├── countries-chart.tsx	← Graphique barres ROA par pays
│ │ ├── countries-table.tsx	← Tableau classement pays
│ │ ├── asn-table.tsx	← Tableau ASN d'un pays
│ │ └── prefixes-table.tsx	← Tableau préfixes d'un ASN
│ └── layouts/	
│ └── root-layout.tsx	← Header + nav + Outlet (wrapper pages)
└── pages/	
│ ├── home-page.tsx	← Dashboard : stats + carte + chart + tableau
│ └── country-page.tsx	← Détail pays : stats + chart opérateurs + tableau ASN

asn-detail-page.tsx	← Fiche ASN : scorecard + pie ROA + tableau préfixes
search-page.tsx	← Résultats de recherche
about-page.tsx	← Explication MANRS/RPKI + sources
App.tsx	← Routes React Router
main.tsx	← Point d'entrée (render React)
index.css	← Thème dark + variables shadcn + fix Leaflet z-index

Fichiers clés à comprendre

lib/api.ts — Le client API. Toutes les pages passent par ces fonctions :

```
api.getStats()           → GET /api/stats
api.getCountries()       → GET /api/countries
api.getCountry("BJ")     → GET /api/countries/BJ
api.getAsn(28683)        → GET /api/asn/28683
api.search("MTN")        → GET /api/search?q=MTN
```

La variable `VITE_API_URL` configure l'URL du backend (défaut : `http://localhost:8000`).

hooks/use-fetch.ts — Hook réutilisable qui retourne `{ data, loading, error }`. Toutes les pages l'utilisent :

```
const { data, loading, error } = useFetch(() => api.getStats())
```

lib/countries.ts — Helpers pour les couleurs ROA : `- roaColorHex(pct) → "#ef4444"` (rouge si `<25%`), `"#f59e0b"` (orange 25-60%), `"#22c55e"` (vert `>60%`)

Points sensibles

- **z-index carte Leaflet** : dans `index.css`, `.leaflet-container { z-index: 0 !important; }` sinon la carte passe au-dessus du header sticky au scroll.
- **Filtre dark mode Leaflet** : le `.leaflet-tile-pane { filter: ... }` inverse les couleurs des tuiles OSM pour un rendu dark. Si on enlève cette ligne, la carte sera en blanc (clair).
- **tsconfig.app.json** : le `"ignoreDeprecations": "6.0"` est nécessaire car TypeScript 6 considère `baseUrl/paths` comme deprecated mais shadcn/ui les requiert.
- **Composants ui/** : générés par `npx shadcn@latest add <nom>`. Ne pas modifier manuellement.
- **VITE_API_URL** : variable d'environnement frontend. En dev elle n'est pas définie (fallback `localhost:8000`). En prod, la définir dans un `.env` ou dans Vercel.

Commandes pour ce module

```
cd frontend
```

```
# Installer
```

```
npm install
```

```
# Développement (hot reload)
```

```
npm run dev -- --port 3000
```

```
# Vérifier les types
npx tsc --noEmit

# Build production
npm run build

# Ajouter un composant shadcn
npx shadcn@latest add <nom-du-composant>
```

Critère de validation

Les 5 pages fonctionnent. La carte s'affiche. La fiche ASN est complète. La recherche retourne des résultats.

MODULE 5 — Déploiement

Objectif

Tout lancer avec `docker compose up` et déployer en ligne.

Fichiers

Fichier	Rôle	Statut
<code>docker-compose.yml</code>	Orchestre DB + API + collecteur + frontend	☐ Existe (manque frontend)
<code>backend/Dockerfile</code>	Image Python pour API et collecteur	☐ Existe
<code>backend/requirements.txt</code>	Dépendances Python	☐ Existe
<code>frontend/Dockerfile</code>	Image Node pour build + nginx pour servir	☐ À créer
<code>README.md</code>	Instructions d'installation	☐ À créer

docker-compose.yml — Services

Service	Image	Port	Dépend de
db	postgres:15	5432	—
api	./backend (Dockerfile)	8000	db (healthcheck)
collector	./backend (commande différente)	—	db (healthcheck)
frontend	./frontend (Dockerfile)	3000	—

Variables d'environnement

Variable	Où	Valeur
DATABASE_URL	API + collecteur	postgresql://manrs:changeme@db:5432
MANRS_API_KEY	Collecteur	Clé obtenue sur observatory.manrs.org
VITE_API_URL	Frontend (build)	URL publique de l'API en prod

Points sensibles

- **db healthcheck** : l'API et le collecteur attendent que PostgreSQL soit prêt avant de démarrer (condition: service_healthy). Sans ça, ils crashent car la DB n'est pas encore prête.
- **Schema auto-init** : le fichier schema.sql est monté dans /docker-entrypoint-initdb.d/ de PostgreSQL. Il s'exécute automatiquement au premier démarrage seulement. Si la DB existe déjà (volume pgdata), le script n'est pas relancé.
- **Volume pgdata** : les données PostgreSQL persistent entre les redémarrages. Pour repartir de zéro : `docker compose down -v` (supprime le volume).
- **Le collecteur et l'API partagent le même Dockerfile** mais avec des commandes différentes : l'API lance `uvicorn`, le collecteur lance `python -m collector.main`.

Commandes pour ce module

```
# Lancer toute la stack
docker compose up -d

# Voir les logs
docker compose logs -f collector
docker compose logs -f api

# Reconstruire après modification du code
docker compose up -d --build

# Repartir de zéro (supprime la DB)
docker compose down -v

# Vérifier que tout tourne
docker compose ps
curl http://localhost:8000/api/stats
```

Critère de validation

`docker compose up` lance tout. L'API répond sur :8000. Le frontend s'affiche sur :3000. Le collecteur log des données.

Dépendances entre modules

Module 1 (DB)

↓

Module 2 (Collecteur) ← nécessite Module 1

↓

Module 3 (API) ← nécessite Module 1 (lit la DB remplie par Module 2)

↓

Module 4 (Frontend) ← nécessite Module 3 (appelle l'API)

↓

Module 5 (Déploiement) ← nécessite tous les modules

Ordre de développement recommandé : 1 → 2 → 3 → 4 → 5

On peut développer le Module 3 (API) en parallèle du Module 2 (Collecteur) si on insère des données de test manuellement dans la DB.

Comptes et accès

Service	Email	Mot de passe	Note
MANRS Observatory	franckzinsou06@gmail.com	ManrsObs2026!	Clé API expire tous les 3 jours

Renouveler la clé API MANRS

```
# 1. Login
curl -s -X POST https://observatory.manrs.org/api/v2/auth/login \
  -H "Content-Type: application/json" \
  -d '{"login":{"email":"franckzinsou06@gmail.com","password":"ManrsObs2026!"}}' \
  | python3 -m json.tool
# → copier le champ "key"

# 2. Créer une nouvelle clé permanente
curl -s -X POST https://observatory.manrs.org/api/v2/keys \
  -H "Authorization: Bearer <token-du-login>" \
  -H "Content-Type: application/json" \
  -d '{"key":{"description":"MANRS Observatory"}}' \
  | python3 -m json.tool
# → copier le champ "key" = la clé API
```